Event-Driven Architecture (EDA)

Software Architecture Notes — Knowledge Base

1. What is it?

Event-Driven Architecture is a design paradigm where system components communicate by producing and consuming events — immutable records that describe something that happened (e.g., OrderPlaced, PaymentFailed). Producers do not know or care who consumes their events, which decouples services in both space and time.

2. Key Building Blocks

- Event Producer — emits events when state changes occur.
- Event Broker/Bus — routes events (e.g., Kafka, RabbitMQ, AWS EventBridge, Google Pub/Sub).
- Event Consumer — subscribes to and reacts to events.
- Event Store — durable, ordered log of events (used heavily in Event Sourcing).

3. Common Patterns

Pub/Sub: producers publish to a topic; any number of subscribers can consume independently. Event Streaming: an append-only, ordered log (Kafka-style) that consumers can replay. Event Sourcing: instead of storing current state, the system stores the full sequence of events, and current state is derived by replaying them. CQRS (Command Query Responsibility Segregation) is often paired with event sourcing to separate write and read models.

4. Advantages

- Loose coupling — producers and consumers evolve independently.
- High scalability — consumers can be scaled independently and added without touching producers.
- Natural fit for real-time processing, analytics, and audit trails.
- Improves resilience — a consumer being temporarily down doesn't block the producer.

5. Challenges

- Eventual consistency — consumers see state changes with a delay.
- Debugging is harder — tracing a business flow across many asynchronous events requires distributed tracing.
- Guaranteeing exactly-once processing is difficult; systems often design for idempotency instead.
- Event schema evolution needs careful versioning to avoid breaking consumers.

6. Typical Use Cases

- Order processing and inventory systems in e-commerce.
- Real-time fraud detection and monitoring dashboards.
- IoT telemetry ingestion.
- Microservices integration without direct service-to-service calls.